

LKA Invoices

System architecture, data flow, security, privacy & maintenance

Lingeshwar Kaparathi & Associates, Chartered Accountants, Siddipet · Internal GST invoice portal

Live: lingeshwarca-invoice.netlify.app · Code: github.com/adi-repo-ai/ca-invoice-app · Firebase project: ca-invoice-app · October 2026

₹0 / month

Free plans only: Firebase
Spark, Netlify free, GitHub

3 access levels

Owner, Admin (no revenue),
Staff; invite-only

4 functions

Small server jobs for admin
tasks

84 tests

46 unit and 38 security /
server tests

Contents

1. What the portal does
2. Architecture at a glance
3. Tool stack
4. How it works: key flows
5. Data: what is stored and how
6. Invoice life cycle
7. Security
8. Privacy
9. Deployment: from code to live site
10. Limits and cost
11. Future maintenance
12. Troubleshooting
13. Source code map

1. What the portal does

A private web app for the firm's staff to **keep client records, create GST-compliant invoices, and send them** to clients as PDFs by email or WhatsApp. It runs in any modern browser on a phone or computer; there is nothing to install.

Who uses it

Only people added by an ADMIN in Settings → Users (currently two people). Clients never log in; they only receive PDF invoices.

Main jobs

Client list · invoice drafts · issue with automatic numbering (LKA/2026-27/0001) · PDF with UPI QR · email / WhatsApp sharing · payment reminders · receipts · client statements · reports · dashboard with alerts · CSV import/export · activity log · backup.

2. Architecture at a glance

The portal has **no traditional server of its own**. The website runs in the browser and talks directly to Google Firebase for sign-in and data. A few admin-only jobs go through small Netlify server functions that hold the secret admin key.

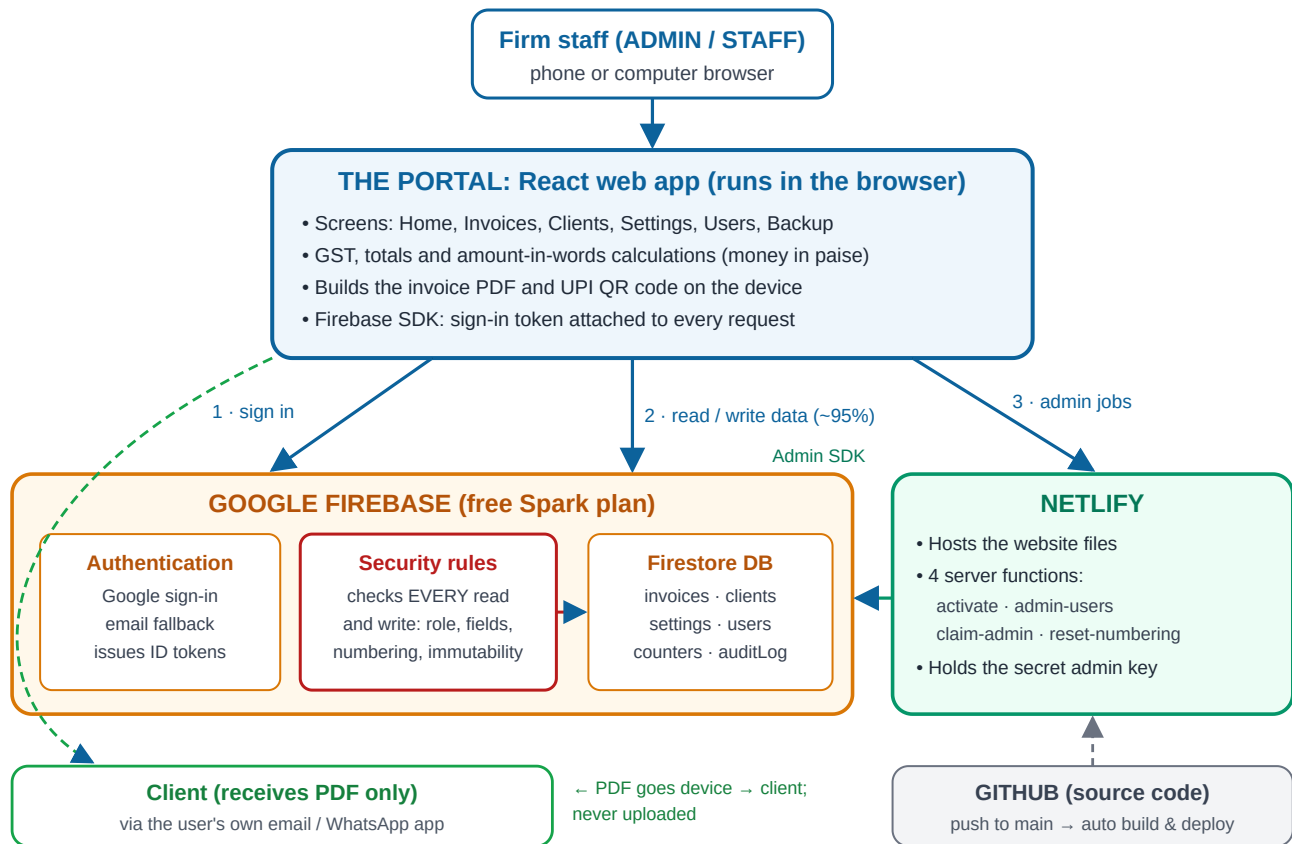


Figure 1: System context. Solid arrows carry data over HTTPS with the user's sign-in token. The PDF goes from the device straight to the client.

3. Tool stack

Layer	Technology	Role in the portal
User interface	React 19 + TypeScript	All screens, forms, dialogs; type-checked code catches mistakes before deploy.
Build	Vite 8	Bundles the code into small static files (HTML, JS, CSS) for fast loading.
Styling	Tailwind CSS v4	Brand colours, responsive phone/desktop layout, day/night theme.
Navigation	React Router 7	Pages such as <code>/invoices</code> , <code>/clients/:id</code> , <code>/settings/users</code> ; admin-only routes are guarded.
PDF & QR	@react-pdf/renderer, qrcode	Creates the A4 invoice PDF and the UPI "Scan to pay" QR code inside the browser.
Sign-in	Firebase Authentication	Google sign-in (main), email + password (fallback) with password reset. Issues short-lived ID tokens carrying the user's role.
Database	Cloud Firestore (NoSQL)	Stores all business data as documents in collections. Queries use indexes from <code>firestore.indexes.json</code> .
Access control	Firestore Security Rules	Server-side rule file (<code>firestore.rules</code>) that approves or rejects every single read and write.

Server jobs	Netlify Functions (Node 22) + Firebase Admin SDK 13	4 small functions for actions a browser must never be trusted with (roles, deleting accounts, numbering reset, first admin).
Hosting / CDN	Netlify	Serves the site over HTTPS worldwide, adds security headers, builds automatically from GitHub.
Source control	GitHub	Full history of every change; <code>main</code> branch = live site.
Testing	Vitest, Firebase Emulators, Playwright	Unit tests for GST maths, numbering, money and words; emulator tests prove the security rules and functions; browser checks for screens.
Admin tooling	Firebase CLI (Cloud Shell)	Deploys security rules and indexes: <code>npx firebase deploy --only firestore:rules,firestore:indexes</code> .

4. How it works: key flows

4.1 Signing in and getting access

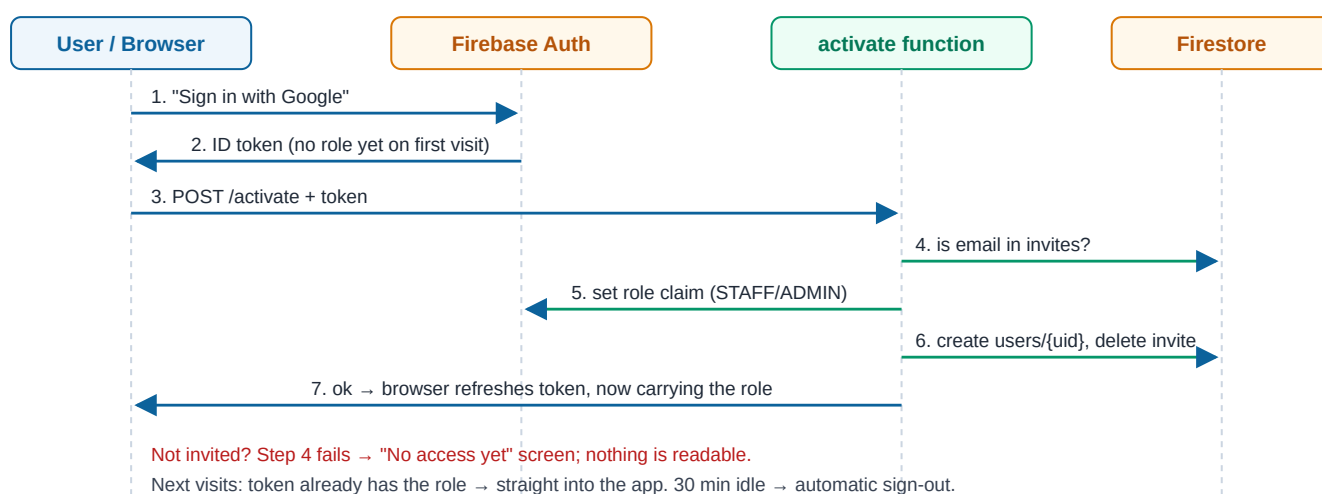
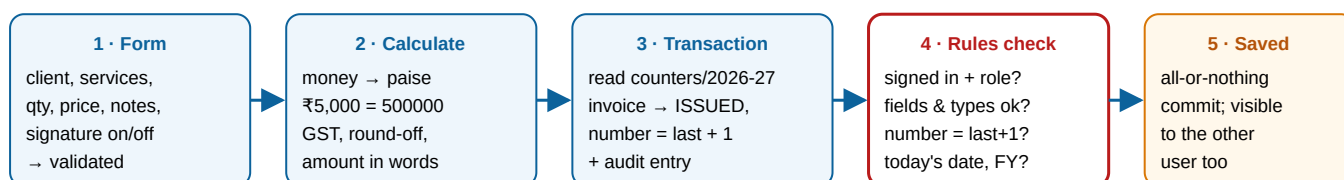


Figure 2: First sign-in of an invited person. The role is stored inside the signed token, so it cannot be faked by the browser.

4.2 Creating and issuing an invoice (direct to Firestore)



If two people issue at the same moment, Firestore lets only one win; the portal automatically retries the other with the next number.

Any failed check → nothing is saved and the user sees "You do not have permission (or the data failed validation)".

Figure 3: Issuing an invoice. Numbers are gap-free and never duplicated because the counter and invoice are written together.

4.3 PDF, email and WhatsApp

- View / Download PDF:** the browser loads the invoice, draws the A4 PDF with your logo, signature (if switched on) and a UPI QR code (unpaid invoices only). It opens in a tab or downloads. **Nothing is uploaded.**
- Send by email / WhatsApp:** on phones the share sheet opens with the PDF attached. On computers the PDF downloads and a ready-written Gmail / mail or WhatsApp chat opens for you to attach it.
- A "sent by email / WhatsApp" entry is written to the invoice's history log.

4. **Payment reminders** (issued invoices): one click opens WhatsApp or email with a polite reminder showing amount, days overdue and UPI ID. Text only, no attachment.
5. **Receipts** (paid invoices): a "RECEIPT" PDF with amount received, TDS and mode; download or send it the same way.
6. **Client statements**: on a client's page, a "STATEMENT" PDF for any period listing every invoice and payment with a running balance.
7. All message wording comes from editable templates in **Settings** → **Goals & messages**. All PDFs (invoice, receipt, statement) are made in the browser and never uploaded.

4.5 Dashboard, alerts and reports (read-only views)

- **Home**: today's overdue / to-collect / tasks / events tiles, nudges (old drafts, very late invoices), 6-month chart, monthly goal ring, weekly summary, recent activity, milestones and a getting-started checklist. Uses cheap server-side sums and counts (about 30 reads per visit).
- **Notification bell**: overdue invoices, tasks due, today's events, stale drafts; refreshes every 5 minutes.
- **Reports** (all staff): per financial year, billed vs received by month, collection rate, outstanding by age, top clients and who owes you; CSV export. Reads each invoice of the chosen year once.
- **Search (Ctrl+K)**: clients by name, invoices by number (e.g. "12"), and pages. **Activity log** (ADMIN): every create / issue / payment / send / cancel / delete, newest first.

4.4 Admin jobs (through Netlify Functions)

Used for: changing roles, disabling / enabling / deleting users, first-admin setup, restarting numbering.

1. The browser sends `POST /.netlify/functions/<name>` with a small JSON body (e.g. `{"action": "delete", "uid": "..."}`) and the header `Authorization: Bearer <ID token>`.
2. The function verifies the token with Google, checks the caller is **ADMIN**, validates the input (e.g. "cannot delete yourself").
3. It acts with the **Admin SDK** using the secret service-account key kept only in Netlify's environment variables.
4. Replies `{"ok": true}` or `{"error": "..."}` , shown as a message in the portal.

5. Data: what is stored and how

Firestore stores **documents** (like small records) grouped in **collections**. Each invoice keeps its own frozen copy of the client and firm details, so later edits to a client or settings never change an already-issued invoice.

Collection	What one document holds	Who can read / write
settings/firm	Firm name, address, GSTIN/PAN, bank & UPI, logo, signature, GST switch & rate, services list, prefix, terms, monthly goal, message templates	All staff read · ADMIN writes
clients	Name, contact person, address, state, GSTIN, PAN, email, mobile / WhatsApp, tags (e.g. ITR, GST), internal notes (never printed)	All staff read, add, edit, delete
invoices	Status, number, dates, client snapshot, firm snapshot, items, reimbursements, totals (paise), amount in words, notes, terms, payment details	All staff; issued invoices locked; numbered ones deletable by ADMIN only
counters/{FY}	Last invoice number used in that financial year (e.g. 2026-27 → 12)	Only moves +1 together with an invoice; reset via function

auditLog	Who did what to which invoice and when: create, edit, issue, payment, cancel, delete, sent	Staff add only; never edited or deleted
users	Email, name, role, revenue & payments permissions, disabled flag, last seen time	ADMIN manages (via functions); user updates own "last seen"
invites	Emails allowed to join, with the role they will get	ADMIN only
tasks, events, notes, links	Personal Home-page items	Owner only, private even from admins
system/bootstrap	One-time lock for the first-admin setup	Server only

Data format details: money is always an integer number of paise (no decimal rounding errors); dates are YYYY-MM-DD in Indian time; financial year runs April–March; invoice numbers look like LKA/2026-27/0001. In transit, the Firebase SDK sends data as encrypted HTTPS (binary protocol); Netlify Functions use JSON over HTTPS.

6. Invoice life cycle

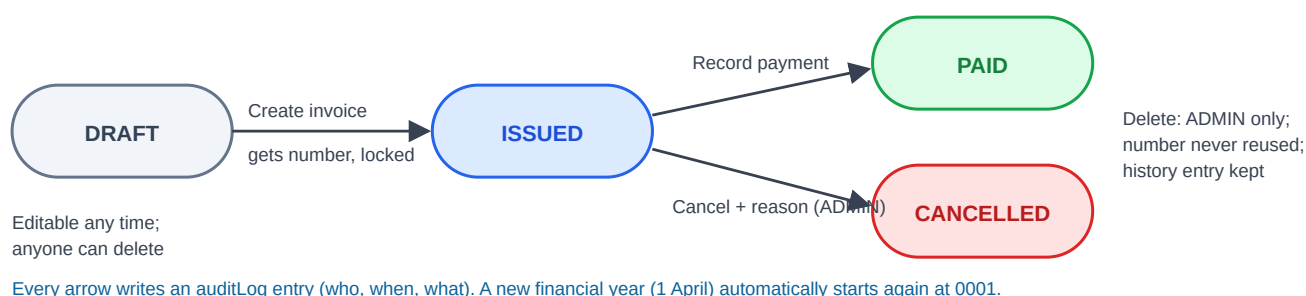


Figure 4: Invoice states. Issued invoices can't be edited, so numbering and amounts stay trustworthy.

7. Security

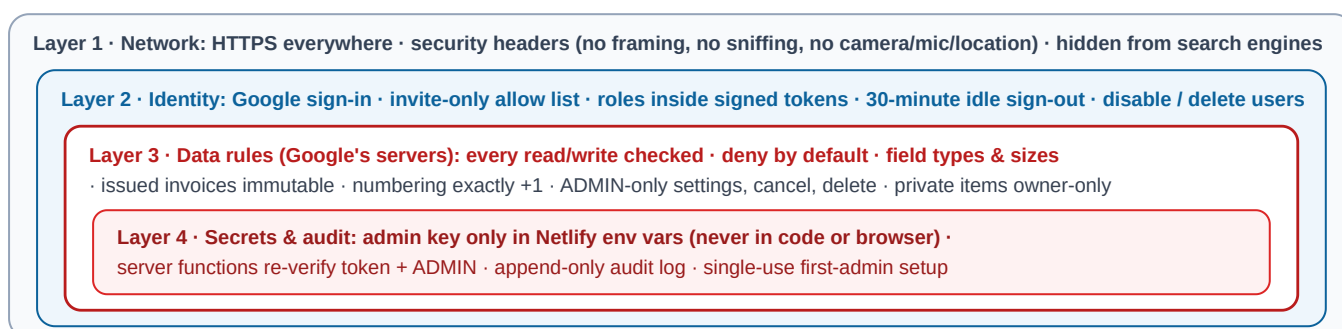


Figure 5: Defence in depth. Even someone who bypasses the website cannot get past layers 2–4.

Threat	How the portal handles it
Stranger finds the URL	Sees only the sign-in page; not on the allow list → "No access yet", zero data readable.
Someone edits browser code	Useless: all checks are enforced by Google's servers (rules) and the functions, not by the browser.
Tampering with an issued invoice	Rules forbid editing issued invoices; only status changes (payment / cancel) are allowed, and each is logged.

Duplicate or skipped numbers	Counter and invoice saved together in one transaction; rules require number = last + 1.
Staff over-reach	STAFF cannot change settings or users, cancel, or delete invoices or clients (enforced by the rules).
Revenue / payment need-to-know	Two per-person permissions in the sign-in token: revenue (totals, Reports, CSV export, Backup) and payments (record payments, receipts, statements). Recording a payment is refused by the rules without it; only an owner (admin with revenue) can change permissions, and other admins cannot change or remove an owner. Individual invoice amounts stay visible to everyone who invoices.
Leaked Firebase web config	Normal and safe: it only identifies the project; access still needs sign-in + rules.
Lost / shared device	30-minute idle sign-out; ADMIN can Disable the account instantly (signs out everywhere).
Admin key exposure	Stored only in Netlify environment variables; never committed to GitHub; never sent to browsers.

8. Privacy

What is stored

Business records only: firm details, client contact & tax IDs, invoices, payments, staff emails/roles, personal Home notes. No passwords are stored by the app (Google handles sign-in).

What is NOT stored or sent

Invoice PDFs (made on the device) · no analytics, tracking or ads · no third-party data sharing · no card/bank login details.

Who sees what

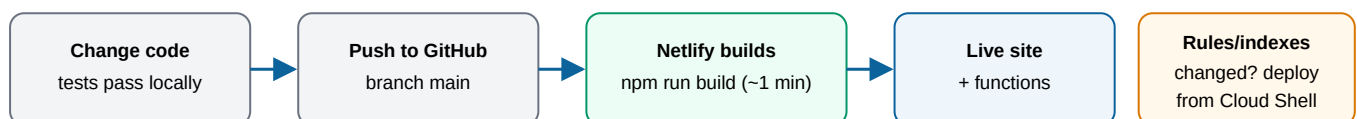
Signed-in firm users see all invoices & clients (shared work). Each person's tasks, events, notes, links are private to them, even from admins.

Where it lives

Your own Firebase project on Google Cloud, in the region chosen when the database was created (Firebase console → Firestore → Settings). Encrypted in transit and at rest by Google.

Your responsibilities: PDFs downloaded or shared by email / WhatsApp are copies outside the portal. Treat them like any client document. Sign out on shared computers. Remove leavers promptly. Use the backup file carefully: it contains all business data.

9. Deployment: from code to live site



Website + functions deploy automatically on push. Firestore rules/indexes are deployed separately (manual, by design):

```
cd ca-invoice-app → git pull → npx firebase deploy --only firestore:rules,firestore:indexes --project ca-invoice-app
```

Configuration	Where it is set
VITE_FIREBASE_* (web config)	Netlify env vars; public by design, used by the browser.

FIREBASE_PROJECT_ID / CLIENT_EMAIL / PRIVATE_KEY	Netlify env vars (Functions scope); secret admin key.
SETUP_CODE	Netlify env var; only for first-admin setup, then delete.
Firm details, GST, services, prefix	In the portal: Settings (stored in Firestore, no code change needed).

10. Limits and cost

Item	Limit	Practical meaning
Invoices per financial year	9,999	4-digit numbering; about 27 per day, every day. Restarts each April.
Clients	No app limit	Tens of thousands fit comfortably.
Lines per invoice	50 + 30	50 service lines and 30 reimbursement lines.
Firestore storage	1 GB	≈150,000–200,000 invoices including history.
Daily reads / writes / deletes	50k / 20k / 20k	Normal day uses a few hundred to ~2,000 reads. Resets ~1:30 pm IST. Over-limit = temporary error, never a charge .
Cost	₹0	Firebase Spark (no card attached), Netlify free tier, GitHub free. No Blaze-plan features are used.

11. Future maintenance

11.1 Routine checklist

How often	Task	Who
Monthly	Settings → Backup → download and store safely (Drive / USB). The free plan has no automatic backups.	ADMIN
Monthly	Review Settings → Users: remove leavers, check roles, keep 2 admins.	ADMIN
Quarterly	Firebase console → Firestore → Usage: confirm well under limits.	ADMIN
Every 6 months	Update libraries (<code>npm outdated</code> , <code>npm update</code>), run <code>npm run test:all</code> and <code>npm run build</code> , then push.	Developer
Each April	Nothing needed: numbering restarts automatically for the new FY. Optionally check the Settings prefix and terms.	Automatic
When GST rules / rate change	Settings → Invoicing: GST switch and rate. If registered for GST, consider restoring "Tax Invoice" title and CGST/SGST split (code change in <code>src/pdf/InvoicePdf.tsx</code>).	ADMIN / Developer

11.2 How to make a change safely

- Edit code in a copy of the GitHub repo (or ask a developer / AI assistant). Most visual changes: `src/pages/*` ; PDF layout: `src/pdf/InvoicePdf.tsx` ; GST maths: `src/lib/tax.ts` ; permissions: `firestore.rules` .
- Run checks: `npx tsc -b` (types), `npm run test:all` (84 tests incl. security), `npm run build` .

3. Try it locally with emulators (`npm run seed:emulator` , demo data, no real data touched).
4. Push to `main` → Netlify deploys automatically. If `firestore.rules` or indexes changed, also deploy them from Cloud Shell (section 9).
5. To undo a bad release: Netlify → Deploys → pick the previous deploy → **Publish deploy** (instant rollback).

11.3 Restoring data

The backup is a JSON file of settings, clients, invoices, counters and audit log. There is no one-click restore screen; restoring is done by a developer with a small script using the Admin key. Keep backups in a safe, private location.

11.4 Possible future enhancements

- Custom domain (e.g. `invoices.yourfirm.in`): about ₹800–1,200/year for the domain; Netlify provides HTTPS free.
- Automatic email sending from the portal and login alerts (need an email service such as Brevo free tier + a Netlify function).
- Recurring invoices for retainer clients; mobile app experience (installable app, bottom navigation, swipe actions).
- One-click restore from backup; scheduled automatic backups; Excel export for Tally.
- Client portal, online payments (Razorpay / Cashfree), multi-branch, e-invoicing (IRN) if GST turnover requires it.

12. Troubleshooting

Symptom	Likely cause → fix
"You do not have permission... or data failed validation"	Rules on Firebase are older than the code → Cloud Shell: <code>git pull</code> then deploy rules. Or the action needs ADMIN.
"The query requires an index"	Indexes not deployed → deploy <code>firestore:indexes</code> from Cloud Shell; wait a few minutes.
"No access yet" after Google sign-in	Email not in Settings → Users, or typed differently → add exactly that Google email.
Admin actions fail with server / token error	Netlify env vars missing or wrong (<code>FIREBASE_*</code>) → fix in Netlify, redeploy.
Site shows old version	Hard refresh (Ctrl+Shift+R); check Netlify Deploys finished.
Errors late in the day, then fine next day	Free daily quota reached (rare) → wait for reset (~1:30 pm IST).
QR code missing on PDF	No UPI ID in Settings, or invoice already paid / cancelled (by design).

13. Source code map

Path	Purpose
<code>src/pages/</code>	Screens: Home, Invoices, InvoiceEdit, InvoiceView, Clients, ClientEdit/View, Reports, Settings, Users, Activity, Backup, Login
<code>src/components/</code>	Shared pieces: layout & menu, buttons, dialogs, client picker, invoice table, icons, theme toggle

src/data/	All database reads/writes: invoices (issue, pay, cancel, delete, counts), clients, settings, audit log, home items
src/lib/	Pure logic with unit tests: GST/tax, financial year & numbering, money (paise), amount in words, validation, CSV, defaults
src/pdf/	Invoice PDF layout and generation (QR, download, share)
src/auth.tsx, src/firebase.ts, src/api.ts	Sign-in state & idle logout; Firebase connection; calls to Netlify functions
netlify/functions/	activate, admin-users, claim-admin, reset-numbering (+ shared token checks)
firestore.rules, firestore.indexes.json	Security rules and query indexes (deploy from Cloud Shell)
tests/emulator/, src/lib/*.test.ts	Security-rule, function and logic tests
scripts/	create-admin, seed-emulator (demo data) helpers
netlify.toml, README.md, docs/	Build/hosting config & security headers; setup guide; PDF guides